

VITyarthi | Open Source Software

Open Source Audit of GIT

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

	Registration Number
Animesh Daiman	24BCY10082
Slot	Date of Submission
E11	25-3-2026

1. Introduction

In today's digital world, much of the software we depend on is not owned by a single company. It is developed openly by communities around the globe. Open-source software is essential in modern computing, from web servers and programming tools to operating systems and mobile platforms. Unlike proprietary software, it lets users study, modify, and share the code. This encourages innovation and collaboration.

This project focuses on understanding the deeper aspects of open-source software rather than just its functionality. For this purpose, the selected software is **Git**, a widely used distributed version control system. Git is essential in modern software development as it helps developers track changes, collaborate efficiently, and manage large codebases.

The objective of this project is to perform a structured audit of Git by analyzing not only what it does, but also why it was created, what problem it solved, and how its open-source license provides freedom to users. The project also explores the philosophical values behind open source, such as transparency, collaboration, and shared ownership of knowledge.

In addition to the theoretical analysis, this project includes five shell scripts that demonstrate practical Linux skills. These scripts are designed to show how open-source tools operate at the command line and how automation is used in real-world environments. Through both conceptual understanding and hands-on implementation, this project should ensure making a complete view of open-source software and its significance.

2. Choosing Your Software

For this project, the chosen open-source software is **Git**. Git is a distributed version control system that allows developers to track changes in their code, collaborate with others, and manage different versions of a project efficiently.

Git was selected because it is widely used in the software industry and has an importance in modern development workflows. It enables multiple developers to work on the same project without conflicts by maintaining version history and supporting branching and merging features.

Another reason for choosing Git is its strong open-source foundation. It is maintained by a global community and is used alongside platforms like GitHub and GitLab, achieving highly relevant for understanding both technical and collaborative aspects of open-source software.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	GPL v2	<i>The foundation everything else runs on</i>
Apache HTTP Server	Web Server	Apache 2.0	<i>Powers ~30% of all websites globally</i>
MySQL	Database	GPL v2 / Commercial	<i>A license with a dual-model story to tell</i>
Firefox	Web Browser	MPL 2.0	<i>A nonprofit fighting for an open web</i>
VLC Media Player	Multimedia	LGPL / GPL	<i>Plays anything — built by students in Paris</i>
LibreOffice	Office Suite	MPL 2.0	<i>Born from a community fork — a real lesson</i>
Python	Programming Language	PSF License	<i>A language shaped entirely by community</i>
Git	Version Control	GPL v2	<i>The tool Linus built when proprietary failed him</i>

Your choice

Write your chosen software here before submission:
Git_____

3. Project Report Structure

Your report should be 12 to 16 pages long. Each section below maps to a unit in the course. Do not pad with screenshots — every page should contain original thinking and analysis written in your own words.

Part A — Origin and Philosophy (Units 1 & 2)

A1: Origin of GIT

Git was created in 2005 by Linus Torvalds during the development of the Linux kernel. At that time, developers were using a proprietary version control system called BitKeeper. However, due to licensing conflicts, the Linux community lost access to this tool. This created a major problem because there was no efficient system to manage contributions from thousands of developers working on the Linux kernel.

The main challenge was handling large-scale collaboration while maintaining speed and reliability. Existing version control systems were either too slow or required constant connection to a central server. This made development inefficient and prone to conflicts.

To solve this, Linus Torvalds designed Git as a distributed version control system. Modern systems, Git allows every developer to have a complete copy of the repository, enabling offline work and faster operations. It was built with a focus on performance, data integrity, and support for non-linear development through branching and merging.

Git was made open source to ensure that the community could freely use, modify, and improve it. This decision reflects the philosophy of collaboration and transparency that defines open-source software.

One of the most impressive aspects of Git is that it was created in only a few weeks. Linus Torvalds prioritized performance, distributed architecture, and data integrity. Git uses a content-addressable storage system, where each file and commit is uniquely identified using cryptographic hashing (SHA-1). This design ensures reliability and prevents data corruption.

A2: License Analysis

Git is released under the GNU General Public License version 2 (GPL v2). This license is designed to ensure that software remains free and open for all users.

The GPL license provides four essential freedoms:

- The freedom to run the program for any purpose
- The freedom to study how the program works
- The freedom to modify the program
- The freedom to distribute copies of the software

Git's license allows users to modify and distribute the software, but with one important condition: any modified version must also be released under the same GPL license. This ensures that improvements remain open to the community.

A company can use Git internally and even modify it, but if they distribute the modified version, they must share the source code. This is different from permissive licenses like MIT, which allow proprietary modifications.

In my opinion, GPL is suitable for Git because it protects the openness of the software while encouraging collaboration. It ensures that no company can take control of the project and make it closed-source.

A3: Ethics of Open Source in Cybersecurity

Open-source software is not just about code; it represents a philosophy of sharing knowledge and working together. Git is a strong example of this philosophy because it enables collaboration among developers across the world.

However, not all software should necessarily be open source. Some applications involve sensitive data or security concerns, where full openness may create risks. Therefore, a balance must be maintained between openness and responsibility.

Companies like Red Hat and Google build successful businesses using open-source software. They contribute back to the community while also creating commercial products. This shows that open source and business can coexist.

The idea of "standing on the shoulders of giants" means that developers build upon existing work instead of starting from scratch. Git enables this by allowing developers to collaborate, reuse code, and improve projects continuously. In this way, open source accelerates innovation rather than limiting it.

In cybersecurity, open source provides transparency, allowing vulnerabilities to be identified and fixed quickly. However, it also means attackers can study the same code. Therefore, security in open source depends on active community review rather than secrecy.

Part B — System Footprint (Unit 2)

Git is typically installed on a Linux system using package managers such as dnf (Fedora) or apt (Ubuntu). The installation process is simple and does not require compilation from source in most cases.

The main binary for Git is located at:

```
/usr/bin/git
```

Git does not run as a background service. Instead, it is a command-line tool that executes when the user runs commands. It operates under the current user account, which ensures that repository access is controlled by file permissions.

Git stores data inside a hidden `.git` directory within each repository. This directory contains all version history, configuration files, and metadata.

Updates for Git are managed through the system's package manager. The open-source community regularly releases updates, including security patches and performance improvements.

On a Linux system (Fedora), Git is installed using the package manager (`dnf install git`). The executable binary is typically located at `/usr/bin/git`, which is part of the system PATH.

All repository data is stored inside a hidden `.git` directory, which contains commits, configuration files, and metadata.

To communicate securely, Git uses standard Linux libraries, especially libpcap and integrates with SSH, which is one of the many tools available in Linux. Additionally, Git interacts with the permissions associated with the different files in relation to the way they are accessed—R/W/X; therefore Git requires a specific level of access to maintain repositories properly.

While working on this project, I understood how Git actually manages version history and how shell scripting can automate system-level tasks efficiently.

Part C — The FOSS Ecosystem (Units 3 & 4)

Git is an essential part of the current open-source software environment. It is frequently used in conjunction with sites like GitHub, GitLab and Bitbucket, where users can also take advantage of other features such as problem tracking and collaboration tools.

Git depends on standard system libraries and tools but does not require heavy external dependencies. It works closely with development environments, compilers, and continuous integration systems.

Git has enabled a large ecosystem of tools and platforms. For example:

- GitHub allows developers to host and share projects
- GitLab supports DevOps pipelines
- Many open-source projects rely on Git for version control

In the context of web development and the LAMP stack, Git is used to manage application code and track changes across deployments.

Git is supported by a large global community. Developers contribute through mailing lists, forums, and open repositories. This strong community ensures continuous improvement and long-term sustainability.

In a LAMP stack (Linux, Apache, MySQL, PHP), Git is used to manage application source code. Developers use Git to track changes, collaborate on features, and deploy updates efficiently. It has an important role in DevOps workflows by integrating with CI/CD pipelines.

Part D — Open Source vs Proprietary (Critical Analysis)

Feature	Git (Open Source)	BitKeeper (Proprietary)
Cost	Free	Paid License Required
Source Code Access	Fully Open	Closed Source
Security Audit	Publicly Auditable	Restricted to company
Modification Freedom	Fully modifiable	Not allowed
Community Support	Large global community	Limited vendor support
Development Model	Community-driven	Company-controlled
Collaboration	Distributed (offline support)	Centralized model

Git provides greater flexibility, transparency, and scalability due to its open-source nature. It enables distributed collaboration and community innovation, whereas BitKeeper restricts access and control to a single organization.

Git is a highly reliable and efficient tool that has become essential in modern software development. Its open-source nature allows developers to collaborate, innovate, and improve projects continuously. The ability to track changes and manage versions makes it a powerful tool for both individuals and organizations.

Based on this analysis, I would recommend Git for real-world use because of its flexibility, performance, and strong community support. I would also consider contributing to open-source projects using Git, as it provides an opportunity to learn, collaborate, and be part of a global development community.

4.Shell Script Tasks

In this project, five shell scripts were developed to demonstrate fundamental shell scripting concepts on a Linux system. Each script performs a specific task such as displaying system information, checking software installation, analyzing directories, processing log files, and generating user-based output.

These scripts were executed in a Linux environment (Fedora) to ensure compatibility with standard Unix commands. The main objective of these scripts is not complexity, but clarity and correct usage of shell scripting concepts such as variables, loops, conditionals, and command substitution.

Script 1 — System Identity Report (Unit 1 & 2)

Purpose:

This script generates a structured report displaying essential system and user information such as OS distribution, kernel version, user name, uptime, and current date.

Concepts Used:

- Variables
- Command substitution (\$())
- if condition
- File reading (/etc/os-release)
- Output formatting using echo

Starter structure

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Animesh Daiman | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Animesh Daiman"
ROLL_NO="24BCY10082"
SOFTWARE_CHOICE="Git"

# --- System Info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p 2>/dev/null || uptime)
DATE=$(date)

# Get OS name
if [ -f /etc/os-release ]; then
    DISTRO=$(grep '^PRETTY_NAME=' /etc/os-release | cut -d= -f2 | tr -d "'")
else
    DISTRO="Unknown Linux Distribution"
fi
```

```
# --- Display ---
echo "=====
echo "      System Identity Report"
echo "=====
echo "Student Name      : $STUDENT_NAME"
echo "Roll Number       : $ROLL_NO"
echo "Software           : $SOFTWARE_CHOICE"
echo "OS Distribution    : $DISTRO"
echo "Kernel Version     : $KERNEL"
echo "User               : $USER_NAME"
echo "Home Directory     : $HOME_DIR"
echo "Uptime             : $UPTIME"
echo "Date & Time        : $DATE"
echo "License            : GNU General Public License v2 (GPL v2)"
echo "=====
```

Script Output:

```
rachit07@gunnus-vivobook:~/ui-navigator$ nano S1.sh
rachit07@gunnus-vivobook:~/ui-navigator$ chmod +x S1.sh
rachit07@gunnus-vivobook:~/ui-navigator$ ./S1.sh
=====
      System Identity Report
=====
Student Name      : Animesh Daiman
Roll Number       : 24BCY10082
Software          : Git
OS Distribution   : Fedora Linux 42 (Workstation Edition)
Kernel Version    : 6.19.8-100.fc42.x86_64
User              : ANIMESH
Home Directory    : /home/rachit07
Uptime           : up 1 hour, 17 minutes
Date & Time       : Mon 30 Mar 19:44:15 IST 2026
License          : GNU General Public License v2 (GPL v2)
=====
```

Working:

This script displays basic system information in a structured format similar to a welcome screen. It collects details such as the Linux distribution, kernel version, current user, home directory, system uptime, and current date and time.

The script uses variables to store values obtained from system commands like `uname`, `whoami`, `uptime`, and `date`. Command substitution `$()` is used to capture command outputs and assign them to variables.

It also reads the Linux distribution name from the `/etc/os-release` file. The output is formatted using multiple `echo` statements to improve readability.

This script demonstrates the use of variables, command substitution, and basic output formatting in shell scripting.

This script clearly shows how shell scripting can be applied to real system tasks

Script 2 — FOSS Package Inspector (Unit 2)

Purpose:

This script checks whether a specified package (Git) is installed on the system and displays its details such as version, summary, and license.

Concepts Used:

- if-else condition
- Command substitution
- Command piping (|)
- grep filtering
- case statement

Starter structure

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Animesh Daiman

PACKAGE="git"    # chosen software

echo "Checking package: $PACKAGE"
echo "-----"

# Check if package is installed (Fedora uses rpm)
if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."

    echo "Package Details:"
    rpm -qi $PACKAGE | grep -E 'Name|Version|Summary|License'

else
    echo "$PACKAGE is NOT installed."
fi

echo ""
echo "Philosophy Note:"
# Case statement for description
case $PACKAGE in
    git) echo "Git: a distributed version control system that enables collaboration
and code sharing." ;;
    httpd) echo "Apache: the web server that built the open internet." ;;
    mysql) echo "MySQL: open-source database used in millions of applications." ;;
    firefox) echo "Firefox: a browser that supports an open and free web." ;;
    *) echo "Unknown software." ;;
esac
```

Output:

```
rachit07@gunnus-vivobook:~/ui-navigator$ nano s2.sh
rachit07@gunnus-vivobook:~/ui-navigator$ chmod +x s2.sh
rachit07@gunnus-vivobook:~/ui-navigator$ ./s2.sh
Checking package: git
-----
git is installed.
Package Details:
Name       : git
Version    : 2.53.0
License    : BSD-3-Clause AND GPL-2.0-only AND GPL-2.0-or-later AND LGPL-2.1-or-later AND MIT
Summary    : Fast Version Control System

Philosophy Note:
Git: a distributed version control system that enables collaboration and code sharing.
```

Working:

This script is designed to check whether a specific open-source software package is installed on a Linux system and display its details. In this project, the selected package is Git.

The script first uses an if-else conditional statement along with the `rpm -q` command to verify if the package is installed. If the package exists, it retrieves detailed information such as the package name, version, summary, or license using the `rpm -qi` command. The output is filtered using the `grep` command to display only relevant fields.

If the package is not installed, the script prints a message indicating that the package is not available on the system.

Additionally, the script uses a case statement to print a short description of the software based on its name. This demonstrates how different outputs can be generated depending on input values.

Overall, the script demonstrates the use of conditional logic, case statements, and command piping in shell scripting, making useful for basic system inspection tasks.

This implementation highlights practical usage of Linux commands in automation.

Script 3 — Disk and Permission Auditor (Unit 2)

Purpose:

This script audits important system directories by displaying their permissions, ownership details, and disk usage.

Concepts Used:

- Arrays
- for loop
- if condition
- Command substitution
- Text processing (awk, cut)

Starter structure :

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Animesh Daiman

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "-----"

for DIR in "${DIRS[@]"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

echo ""
echo "Git Binary Check:"
# Check Git location
if [ -f "/usr/bin/git" ]; then
    ls -l /usr/bin/git
else
    echo "Git binary not found"
fi
```

Output:

```
rachit07@gunnus-vivobook:~$ nano s3.sh
rachit07@gunnus-vivobook:~$ chmod +x s3.sh
rachit07@gunnus-vivobook:~$ ./s3.sh
Directory Audit Report
-----
/etc => Permissions: drwxr-xr-x. root root | Size: 36M
/var/log => Permissions: drwxr-xr-x. root root | Size: 466M
/home => Permissions: drwxr-xr-x. root root | Size: 5.7G
/usr/bin => Permissions: dr-xr-xr-x. root root | Size: 753M
/tmp => Permissions: drwxrwxrwt. root root | Size: 16K

Git Binary Check:
-rwxr-xr-x. 1 root root 4566520 Feb  3 05:30 /usr/bin/git
```

Working:

This script audits important system directories by displaying their permissions, ownership, and disk usage. It processes a predefined list of directories such as /etc, /var/log, /home, /usr/bin, and /tmp.

The script uses a for loop to iterate through each directory in the list. For each directory, it first checks whether it exists using an if condition. If the directory is present, it retrieves its permissions and ownership details using the ls -ld command and extracts relevant fields using awk.

The disk usage of each directory is calculated using the `du -sh` command, and the output is refined using the `cut` command. If a directory does not exist, the script prints an appropriate message.

Additionally, the script checks whether the Git binary exists in `/usr/bin/git` and displays its permissions using the `ls -l` command.

This script demonstrates how shell scripting can be used to automate system inspection tasks.

The script effectively demonstrates real-world system interaction using shell scripting.

Script 4 — Log File Analyzer (Units 2 & 5)

Purpose:

This script analyzes a log file to count the number of occurrences of a specific keyword and displays the last matching entries.

Concepts Used:

- Command-line arguments (\$1, \$2)
- Default values (\${var:-default})
- while loop
- if condition
- String matching using grep
- Arithmetic operations

Starter structure

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Animesh Daiman
# Usage: ./script4.sh logfile keyword

LOGFILE=$1
KEYWORD=${2:-"error"} # default keyword
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Check if file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "File is empty."
    exit 1
fi

# Read file line by line and count matches
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

Output:

```
rachit07@gunnus-vivobook:~$ cat <<EOF > mylog.txt
Line 1: INFO - System boot started
Line 2: ERROR - Database connection failed
Line 3: WARNING - High CPU usage detected
Line 4: ERROR - Disk space critical
Line 5: DEBUG - Maintenance mode active
EOF
rachit07@gunnus-vivobook:~$ bash s4.sh mylog.txt
Keyword 'error' found 2 times in mylog.txt

Last 5 matching lines:
Line 2: ERROR - Database connection failed
Line 4: ERROR - Disk space critical
```

Working:

This script analyzes a log file to count how many lines contain a specific keyword. The log file name is provided as a command-line argument, or an optional keyword can also be specified. If no keyword is given, the script uses "error" as the default.

The script first checks whether the file exists using an if condition. It also verifies whether the file is empty before processing it. If the file is missing or empty, the script exits with an appropriate message.

The main logic uses a while read loop to process the file line by line. For each line, it checks whether the keyword is present using the grep command. If a match is found, a counter variable is incremented.

After processing the file, the script prints the total number of matching lines. It also displays the last five lines containing the keyword using a combination of grep and tail.

This script demonstrates how to handle file input, use loops, apply conditional checks, and work with command-line arguments in shell scripting.

This script successfully demonstrates the practical use of shell scripting concepts such as variables, conditionals, or loops in a real-world Linux environment. It executes correctly on a Linux system and produces the expected output without errors.

Script 5 — The Open Source Manifesto Generator (Unit 5)

Purpose:

This script generates a personalized open-source manifesto based on user input and saves it to a text file.

Concepts Used:

- User input (read)
- Variables
- Command substitution (date, whoami)
- String concatenation
- File output redirection (>)
- File display using cat

Starter structure

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Animesh Daiman

echo "Answer three questions to generate your manifesto."
echo ""

# Taking user input
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# Get current date
DATE=$(date '+%d %B %Y')

# Output file
OUTPUT="manifesto_$(whoami).txt"

# Compose paragraph
echo "On $DATE, I reflect on the importance of open source. I regularly use $TOOL, which helps me in my daily work. For me, freedom means $FREEDOM. I believe in building $BUILD and sharing it openly so that others can learn, improve, and collaborate. Open source is not just about code, but about community and knowledge sharing." > "$OUTPUT"

# Alias concept (comment only)
# alias ll='ls -l'

echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat "$OUTPUT"
```

Output:

```
rachit07@gunnus-vivobook:~$ chmod +x s5.sh
rachit07@gunnus-vivobook:~$ ./s5.sh
Answer three questions to generate your manifesto.
1. Name one open-source tool you use every day: GIT
2. In one word, what does 'freedom' mean to you? WINGS
3. Name one thing you would build and share freely: Community helping projects

Manifesto saved to manifesto_rachit07.txt

On 30 March 2026, I reflect on the importance of open source. I regularly use GIT, which helps me in my daily work. For me, freedom means WINGS. I believe in building Community helping projects and sharing it openly so that others can learn, improve, and collaborate. Open source is not just about code, but about community and knowledge sharing.
```

Working:

This script generates a personalized open-source manifesto by taking input from the user. It asks three questions related to open-source usage, the meaning of freedom, and what the user would like to build & share.

The script uses the `read` command to capture user input and store it in variables. It then uses the `date` command to include the current date in the manifesto. A paragraph is created by combining user inputs using string concatenation.

The generated manifesto is saved to a text file using output redirection (`>`). The filename includes the current username to make it unique. Finally, the script displays the saved content using the `cat` command.

A comment is also included to demonstrate the concept of aliases in shell scripting. This script demonstrates how user interaction file handling can be combined to create meaningful output in shell scripting.

This script successfully demonstrates practical usage of shell scripting concepts in a real-world Linux environment.

This script executes successfully on a Linux system produces the expected output without errors.

5. Reference Resources

The following resources were used to understand Git, open-source philosophy, licensing, and Linux shell scripting concepts used in this project:

Git and Version Control

- Git Official Website.
Available at: <https://git-scm.com>
(Used to understand the purpose, features, and history of Git as a version control system)
- Git Documentation.
Available at: <https://git-scm.com/docs>
(Used to study Git commands, workflows, and internal working)
- Chacon, Scott and Straub, Ben. *Pro Git Book*.
Available at: <https://git-scm.com/book/en/v2>
(Used as a primary reference for understanding Git architecture, branching, and distributed version control)
- GitHub Documentation.
Available at: <https://docs.github.com>
(Used to understand practical usage of Git in real-world development and collaboration)

Open Source Philosophy and Licensing

- GNU Project. *The Free Software Definition*.
Available at: <https://www.gnu.org/philosophy/free-sw.html>
(Used to understand the concept of software freedom and open-source principles)
- Open Source Initiative (OSI). *The Open Source Definition*.
Available at: <https://opensource.org/osd>
(Used to study licensing standards and requirements for open-source software)
- Git License Information (GPL v2).
Available at: https://git-scm.com/docs/git#_license
(Used to understand the licensing model under which Git is distributed)
- Eric S. Raymond. *The Cathedral and the Bazaar*.
Available at: <http://www.catb.org/~esr/writings/cathedral-bazaar/>
(Used to understand community-driven development in open source)

Linux and Shell Scripting

- Shotts, William. *The Linux Command Line*.
Available at: <https://linuxcommand.org>
(Used to understand Linux commands and shell scripting fundamentals)
- GNU Bash Manual.
Available at: <https://www.gnu.org/software/bash/manual/>
(Used for scripting concepts such as loops, variables, and conditional statements)
- Linux Manual Pages (man pages).
Accessed using terminal commands such as `man ls`, `man grep`, `man awk`
(Used to understand command usage and options)

Software Licensing and Resources

- SPDX License List.
Available at: <https://spdx.org/licenses/>
(Used to verify license details and classifications)
- Choose a License.
Available at: <https://choosealicense.com>
(Used to compare open-source licenses and their permissions)

Additional Learning Resources

- Fedora Project Documentation.
Available at: <https://docs.fedoraproject.org>
(Used to understand Linux environment, package management, and system structure)
- Online tutorials and practice exercises for Bash scripting
(Used for debugging scripts and improving practical implementation)